

1. Introducción

TypeScript es un lenguaje de programación que añade tipado estático al lenguaje JavaScript, que es de tipado dinámico. Se trata de un superconjunto sintáctico estricto de JavaScript, lo que significa que todo el código JavaScript existente es válido en TypeScript. Por el contrario, el código TypeScript no es válido en JavaScript, ya que aporta características adicionales.

Este tema se presentarán los escollos de trabajar con los tipos dinámicos de JavaScript y se explicará cómo el tipado estático de TypeScript ayuda a detectar errores de forma temprana, lo que aumenta la estabilidad de tu código.

Abordaremos los conceptos esenciales y avanzados de TypeScript necesarios para crear aplicaciones *full-stack*. A lo largo del curso, conocerás las opciones de configuración más comunes del lenguaje, sus tipos más importantes y cómo y cuándo utilizar las características de tipado estático de TypeScript.

Ventajas de TypeScript

TypeScript hace que trabajar con el sistema de tipos de JavaScript sea menos propenso a errores, ya que su compilador nos ayuda a detectar los errores de tipos al instante. Dado que JavaScript es un lenguaje de tipado *dinámico*, no es necesario especificar el tipo de una variable al declararla. En cuanto el tiempo de ejecución ejecuta el *script*, comprueba estos tipos en función de su uso. Sin embargo, esto significa que los errores derivados de tipos no válidos (por ejemplo, llamar a `array.map` sobre una variable que contiene un número en lugar de una matriz) no se detectarán hasta el tiempo de ejecución, momento en el que el programa completo falla.

Con TypeScript, dispones de todas las nuevas características de JavaScript, lo que te permite evitar errores de tipografía antes de que se produzcan. La limitación de TypeScript es que los navegadores no pueden ejecutarlo.

Los navegadores solo entienden JavaScript, por lo que TypeScript debe compilarse a JavaScript. Este capítulo comienza con la configuración básica.

Configuración del primer proyecto

Abre una nueva carpeta en VS Code y crea en ella un archivo `index.html` básico (véase el Listado [1-1](#)). Este ejemplo también hace referencia a un archivo de JavaScript llamado `main.js`.

```
<!DOCTYPE html>
<html lang="en">
<head>
```

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<title>Conceptos básicos de TypeScript</title>
</head>
<body>
  <h1>Conceptos básicos de TypeScript</h1>
  <script src="main.js"></script>
</body>
</html>
```

Listado 1-1: Archivo index.html básico

A continuación, instala TypeScript globalmente en tu sistema con el siguiente comando. Añade «`sudo`» si utilizas un sistema Mac o Linux.

```
1 npm i -g typescript
```

Trabajar con npm

El gestor de paquetes predeterminado para Node.js es **npm**. Allí encontrarás módulos para cualquier tarea, procedentes del registro en línea de <https://www.npmjs.com>. Comprueba que **npm** está disponible en tu equipo local ejecutando lo siguiente en la línea de comandos:

```
$ npm -v
```

Si no aparece ninguna versión o si la versión es anterior a la actual, instala la última versión LTS de Node.js, que incluye **npm**.

Ten en cuenta que en <https://www.npmjs.com> no existe ningún proceso de selección ni control de calidad. Cualquiera puede publicar paquetes, y el sitio depende de la comunidad para que informe de aquellos que sean maliciosos o no funcionen correctamente.

Al ejecutar lo siguiente, se muestra una lista de los comandos disponibles:

```
$ npm
```

Ahora, crea el archivo `main.ts` y añádele un sencillo `console.log`. El navegador solo entiende JavaScript, por lo que debes convertirlo a JavaScript utilizando el comando `tsc main.ts`. El comando `tsc` ejecuta el compilador de TypeScript y convierte el archivo de TypeScript a JavaScript. Este comando creará un nuevo archivo JavaScript llamado `main.js` en el mismo directorio. Véase la Figura [1-1](#).

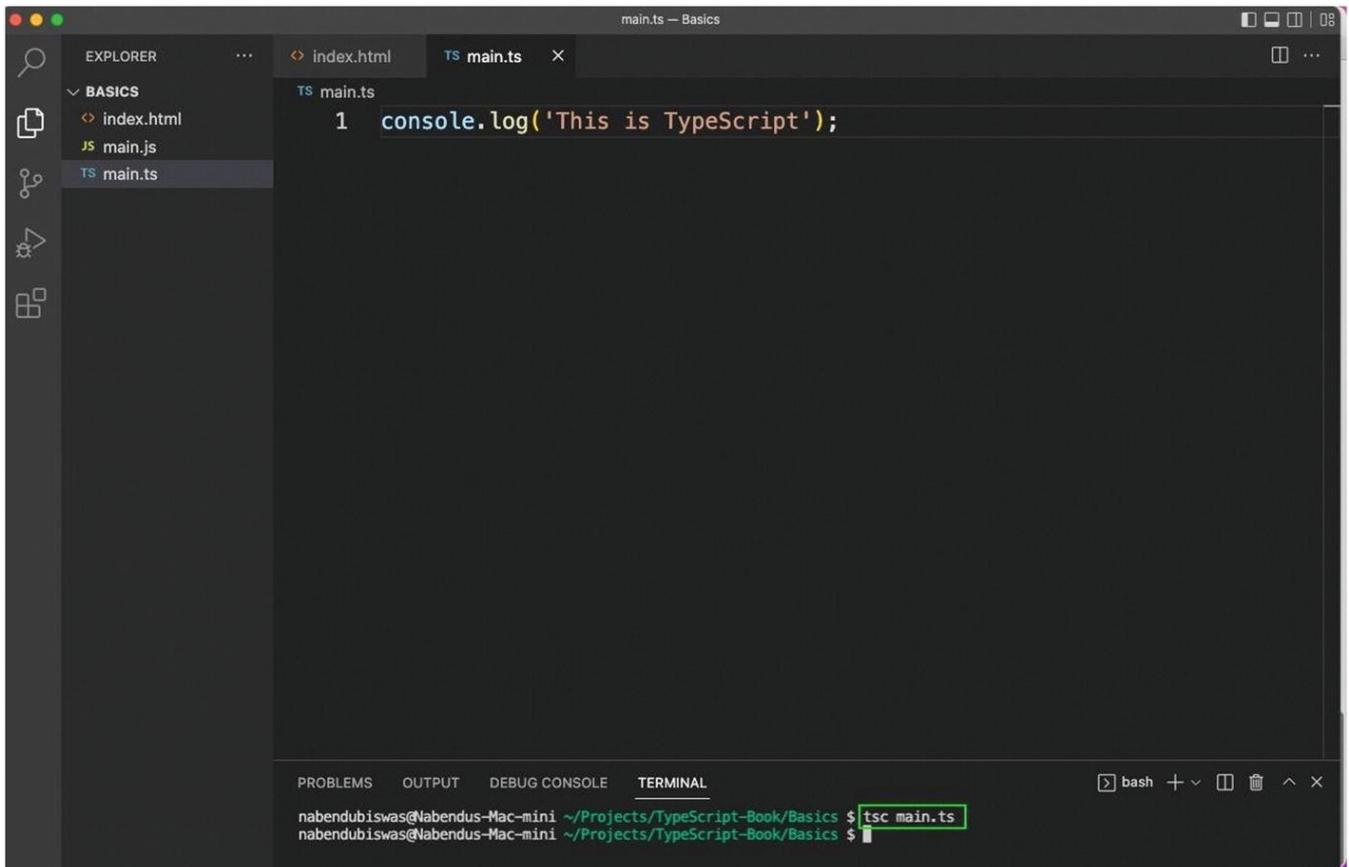


Figura 1-1: main.ts sencillo

En este proyecto también utilizarás la fantástica extensión **Live Server** (véase la Figura [1-2](#)) para no tener que volver a ejecutar el proyecto después de cada cambio.

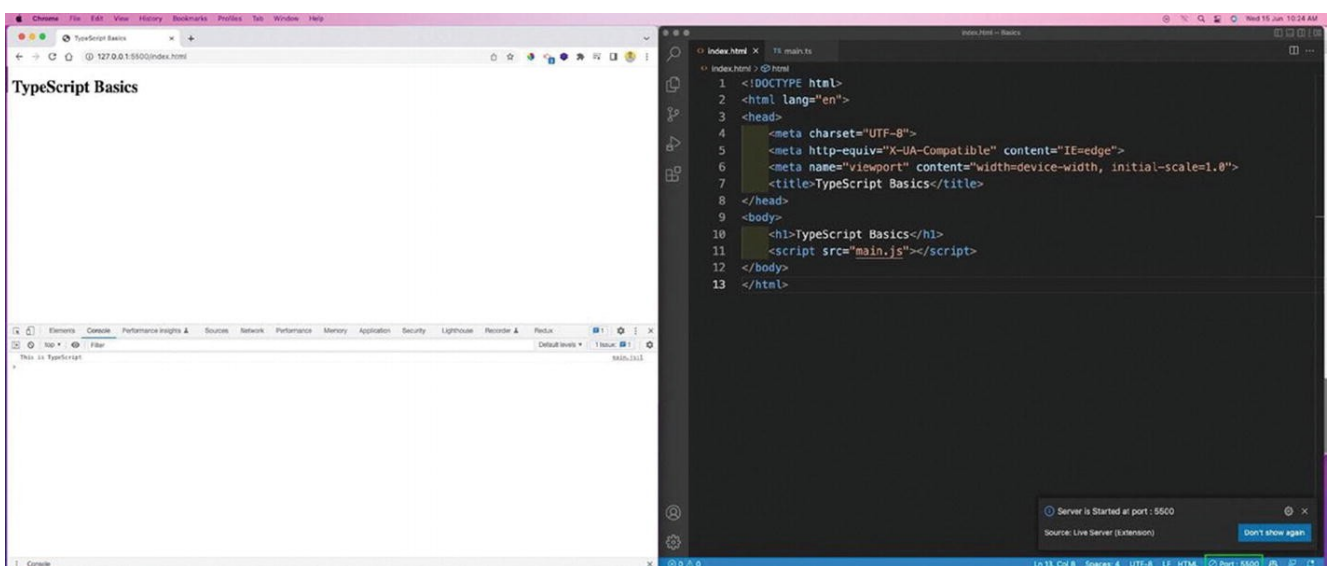


Figura 1-2 Live Server

Resumen: En este capítulo introductorio, has completado la configuración básica de TypeScript. Has instalado TypeScript de forma global, has creado archivos básicos de HTML y TypeScript, y has conseguido que funcionen juntos.

2. Conceptos básicos de TypeScript

Este capítulo marca el inicio de tu andadura con TypeScript. Aquí aprenderás sobre los diferentes tipos y cómo utilizarlos en tus proyectos. Empezarás con el tipo «number», seguido de los tipos «string» y «boolean». También aprenderás sobre muchos otros tipos y terminarás con las interfaces.

El tipo «número»

Crea un archivo `index.ts` en la configuración que has realizado anteriormente. A continuación, añade el código del Listado **2-1** a dicho archivo. Tienes dos variables: `myNum` y `anotherNum`. El código asigna a `anotherNum` el tipo numérico.

Esto significa que las variables `myNum` y `anotherNum` no pueden contener nada que no sean números.

```
//Numbers
let myNum = 10;
let anotherNum: number = 20;
myNum = 12;
myNum = '12';
anotherNum = 30;
anotherNum = false;
```

Listado 2-1 Tipos numéricos

Si pasas el ratón por encima del error, podrás ver los problemas reales, tal y como se muestra en las figuras **2-1** y **2-2**.

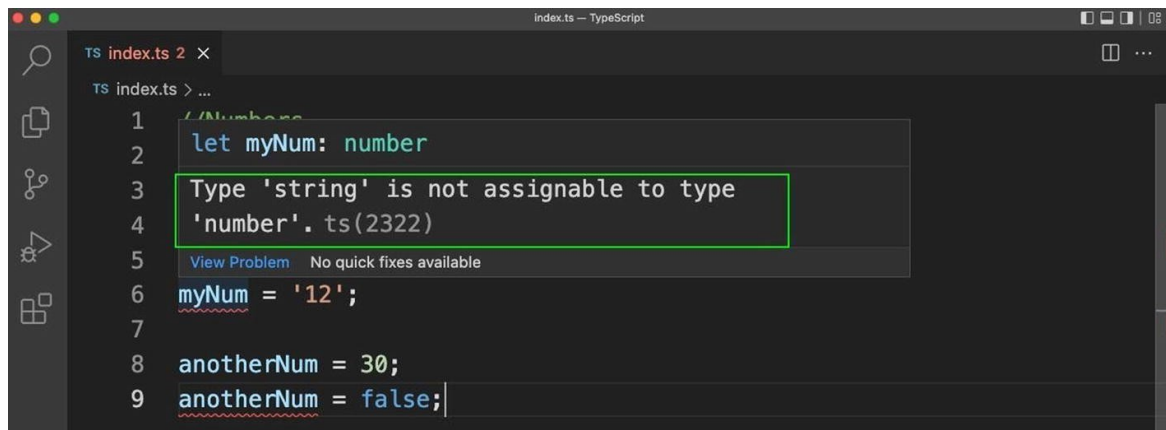


Figura 2-1: Problema con los números

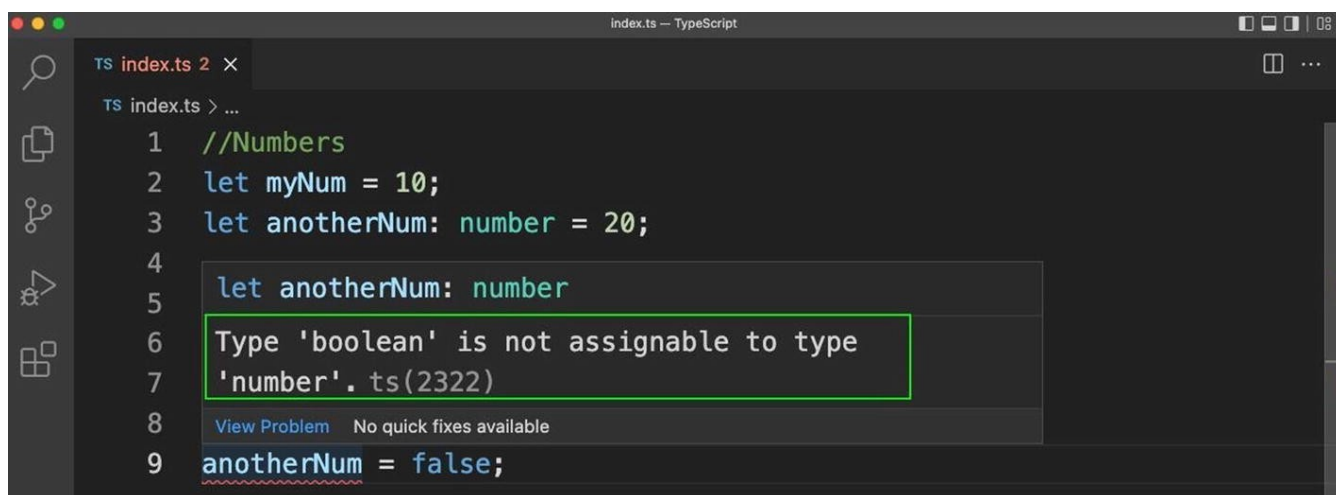


Figura 2-2 Problema con los booleanos

Como puedes ver en este ejemplo, aunque no asignes un tipo, TypeScript lo infiere. Pasa el ratón por encima de `myNum` y se mostrará el tipo numérico. Véase la figura **2-3**.



Figura 2-3 TypeScript

Los tipos String y Boolean

Lo mismo ocurre con los tipos `String` y `Boolean`. Si primero asignas un tipo y luego indicas uno incorrecto, aparecerá un error. En segundo lugar, los tipos se deducen. Esto significa que, si no especificas un tipo, TypeScript asignará uno en función del valor. Véase el Listado **2-2**.

```
//String
```

```
let myStr: string = 'Hello';
let anotherStr = 'World';
myStr = true;
anotherStr = 45;
//Boolean
let myBool: boolean = true;
let anotherBool = false;
myBool = 'true';
anotherBool = 76;
```

Listado 2-2 Tipos «String» y «Boolean»

Inferencia

Así pues, quizá te preguntes cuándo debes asignar un tipo y cuándo es mejor dejar que TypeScript lo asigne automáticamente.

En la mayoría de los casos, debes dejar que sea TypeScript quien asigne el tipo. En el Listado **2-3**, a la variable «salary» no se le asignó ningún tipo.

Más adelante, le asignarás un tipo numérico, de cadena y booleano.

```
//Inferencia

let salary;
salary = 12000;
salary = '12000';
salary = true;
```

Listado 2-3 Tipos incorrectos

Esto no es correcto. Cuando quieras asignar un valor más adelante, deberás especificar un tipo explícito.

Ahora empezarás a obtener errores de tipo, tal y como se muestra en el Listado **2-4**.

```
//Inferencia

let salary:number;
salary = 12000;
salary = '12000';
salary = true;
```

Listado 2-4 Errores de tipo con inferencia

Objetos

En esta sección se explica qué son los objetos en TypeScript. En el ejemplo del Listado [2-5](#), a un objeto que contiene dos cadenas se le asigna un número y un valor booleano.

Si pasas el ratón por encima del objeto, se mostrarán los tipos de datos.

```
//Objetos
const developer = {
  firstName: 'Nabendu',
  lastName: 'Biswas',
  age: 40,
  isTrainer: true
}
```

Listado 2-5 Objeto sin tipos

Ahora crea un nuevo objeto en el que especifiques el tipo de cada clave (véase el Listado [2-6](#)). Aparecerá un error si intentas asignar un valor diferente a una clave o a una clave que no existe. Véase la Figura [2-4](#).

```
const newDeveloper: { name: string; age: number; isDev: boolean } = {
  name: 'Mousam',
  age: 39,
  isDev: true
}

newDeveloper.name = 'Mousam Mishra';
newDeveloper.age = 'Forty';
newDeveloper.firstName = 'Mousam';
```

Listado 2-6 Objeto con tipos

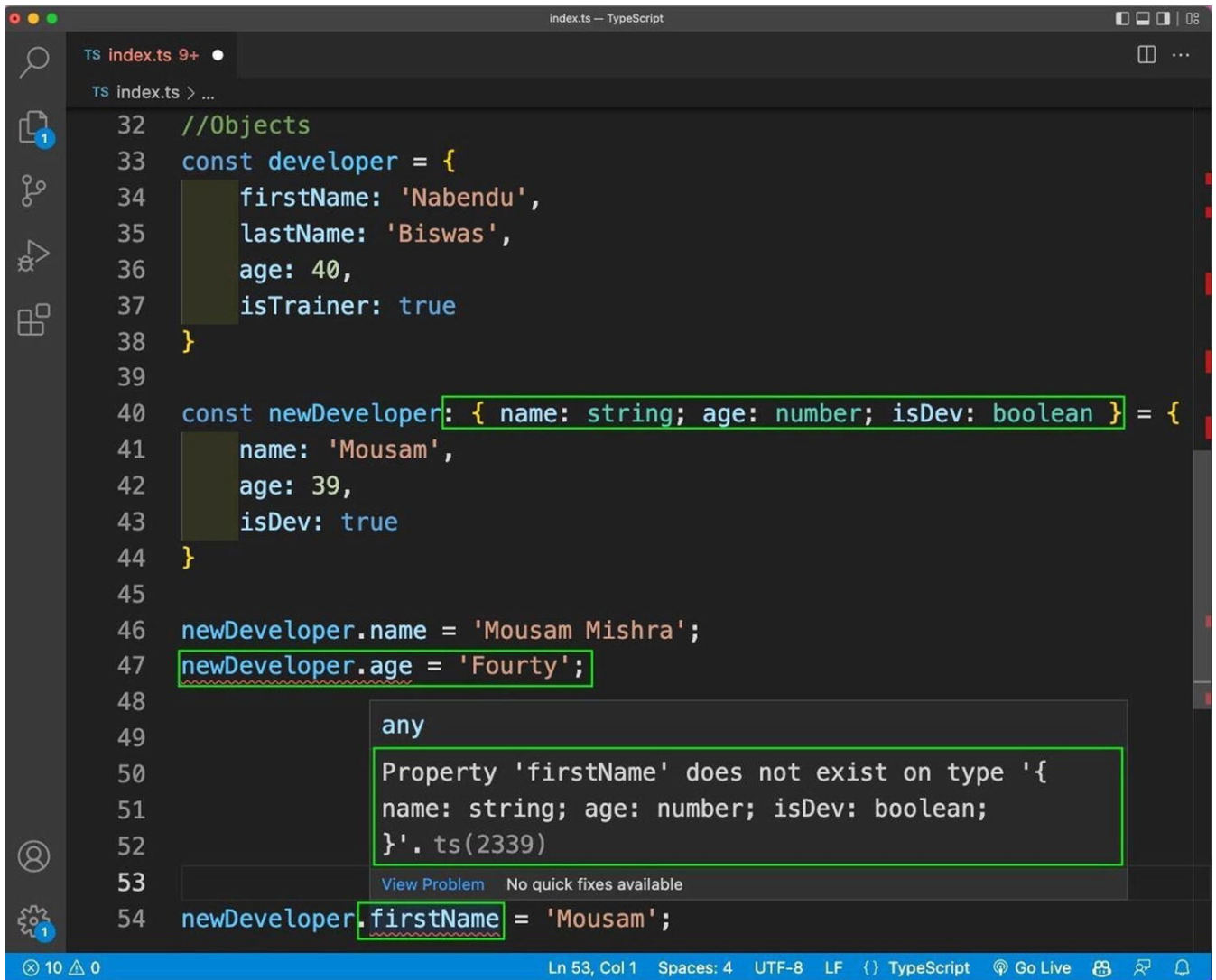


Figura 2-4 Errores en los objetos

Matrices

En esta sección se tratan las matrices. En TypeScript, si se proporciona una matriz de cadenas, como los idiomas del siguiente ejemplo, no se puede añadir un número ni un valor booleano a dicha matriz.

También puedes declarar explícitamente que tienes una matriz de un tipo determinado, como al declarar una matriz de tipos numéricos. Véase el Listado [2-7](#).

```
//Matrices (Arrays)
const languages = ['React', 'Angular', 'Vue'];
languages.push('TypeScript');
languages.push(56);
languages.push(true);
const numbers: number[] = [51, 22, 33];
numbers.push(56);
numbers.push('56');
numbers.push(true);
```

Listado 2-7 Matrices con tipos

Matrices complejas

En esta sección, aprenderás a crear una matriz de objetos. Se especifica el tipo y se indica el tipo de las claves del objeto (véase el Listado **2-8**).

```
const arrOfObj: { name: string; age: number }[] = [  
  { name: 'Nabendu', age: 40 },  
  { name: 'Mousam', age: 39 }  
];  
arrOfObj.push({ name: 'Shikha', age: 39 });  
arrOfObj.push({ name: 'Hriday', age: 'Eight' });
```

Listado 2-8: Matrices complejas con tipos

Si quieres especificar el tipo de la matriz, debes utilizar dos corchetes `[]` en la inferencia de tipos, tal y como se muestra en el Listado **2-9**.

```
const arrOfArrays: number[][] = [  
  [11, 32, 43],  
  [34, 75, 64]  
];  
arrOfArrays.push([21, 32, 13]);
```

Listado 2-9: Matrices más complejas con tipos

Funciones

En esta sección se analizan las funciones a modo de ejemplo. Supongamos que necesitas sumar dos números y crear una función llamada `addNums` para hacerlo.

Si no se especifica el tipo, no se producirá ningún error, aunque se pase una cadena. Véase el Listado **2-10**.

```
//Funciones  
const addNums = (num1, num2) => {  
  return num1 + num2;  
}  
addNums(10, 20);  
addNums(10, '20');
```

Siempre debes especificar el tipo, como en `multiNums`. También es recomendable especificar el tipo de retorno, ya que así se puede detectar el error si se indica un tipo de retorno incorrecto, como en `modNums`.

Si no se especifica un tipo de retorno, se debe indicar «`void`», tal y como se muestra en «`printSum`» del Listado **2-11**.

```
const multiNums = (num1: number, num2: number) => {
    return num1 * num2;
}
multiNums(10, 20);
multiNums(10, '20');
const modNums = (num1: number, num2: number): number => {
    // return num1 % num2;
    return num1 > num2
}
modNums(10, 20);
modNums(10, '20');
const printSum = (num1: number, num2: number): void => {
    console.log(num1 + num2);
}
printSum(10, 20);
printSum(10, '20');
```

Listado 2-11: Tipos de función

Tipos de unión

También puedes crear tipos de unión, en los que una variable puede tener varios tipos. Supongamos que tienes una variable llamada `numOrStr`, que puede ser de tipo `número` o `cadena`.

También puedes tener una matriz que solo pueda contener elementos de tipo `número` o `cadena`. Véase el Listado **2-12**.

```
//Tipos de unión
let numOrStr: number | string;
numOrStr = 10;
numOrStr = 'Ten';
```

```
let arr: (number | string)[] = [10, 'Ten', true];
```

Listado 2-12 Tipos de unión

Tipos literales

Con los tipos literales, solo se especifican los términos aceptables. Por ejemplo, en el tipo `myLiteral`, `Nabendu`, `Mousam`, `Shikha` y `Hriday` son los únicos valores aceptables.

El Listado [2-13](#) utiliza «`Parag`» y el programa devuelve errores.

```
//Tipos literales
let myLiteral: 'Nabendu' | 'Mousam' | 'Shikha' | 'Hriday' = 'Nabendu';
myLiteral = 'Mousam';
myLiteral = 'Shikha';
myLiteral = 'Hriday';
myLiteral = 'Parag';
```

Listado 2-13 Tipos literales

Tipos enumerados

En esta sección se analiza el tipo «`enum`», que es una combinación del tipo unión y el tipo literal. El Listado [2-14](#) muestra un tipo predefinido con la variable «`enum`».

A continuación, podrás utilizarlo en tu código.

```
//Enumeración
enum Role { ADMIN, READ_ONLY, AUTHOR };
const myRole = Role.ADMIN;
const hridayRole: Role = Role.AUTHOR;
```

Listado 2-14 Tipos de enumeración

Tipos opcionales

En esta sección se analizan *los valores opcionales*. Supongamos que quieres un campo «edad», que debe ser un número.

En el ejemplo del Listado [2-15](#), en `optionalObj` la edad se declara como un número y está indefinida. El problema de este enfoque es que hay que dejarla indefinida si no se quiere especificar.

En el ejemplo «`betterOptObj`», la edad se indica con «?», lo que significa que, si se proporciona, debe ser un número, pero no es obligatorio.

```
//Opcionales
let optionalObj: { name: string; age: number | undefined } = {
  name: 'Nabendu',
  age: undefined
};

let betterOptObj: { name: string; age?: number } = {
  name: 'Nabendu'
};
```

Listado 2-15 Tipos opcionales

Interfaces y tipos

En esta sección se analizan las interfaces. Son una forma más adecuada de definir tipos para las diferentes propiedades de un objeto. El Listado **2-16** muestra una interfaz `Developer` que incluye algunas propiedades.

Puedes utilizar esta interfaz en dos variables diferentes, denominadas «person1» y «person2».

```
//Interfaces
interface Developer {
  name: string;
  age: number;
  isDev: boolean;
}

const person1: Developer = {
  name: 'Nabendu',
  age: 40,
  isDev: true
}

const person2: Developer = {
  name: 'Mousam',
  age: 39,
  isDev: true
}
```

Listado 2-16 Interfaces

Los tipos son similares a las interfaces. Como se puede ver en el Listado **2-17**, se utilizan en `DeveloperType`.

Las interfaces solo se pueden utilizar en objetos, mientras que los tipos se pueden utilizar en cadenas, matrices de objetos o cualquier otro elemento.

```
//Tipos
type DeveloperType = {
  name: string;
  age: number;
  isDev: boolean;
}
const person3: DeveloperType = {
  name: 'Nabendu',
  age: 40,
  isDev: true
}
type PersonName = string;
const person4: PersonName = 'Nabendu';
type CoderType = {
  name: string;
  category: 'frontend' | 'backend' | 'mobile';
  age: number;
}[];
const coder1: CoderType = [
  { name: 'Nabendu', category: 'frontend', age: 40 },
  { name: 'Mousam', category: 'backend', age: 39 },
]
```

Listado 2-17 Tipos

Ejecución del código

Dado que el archivo `index.ts` contiene muchos errores, añadirás parte de su contenido al archivo `main.ts`. También debes ejecutar el comando `tsc main.ts` para convertir el código en un archivo JavaScript. Véase el Listado **2-18**.

```
//Tipos
type DeveloperNewType = {
  name: string;
```

```

    age: number;
    isDev: boolean;
}
const person5: DeveloperNewType = {
    name: 'Nabendu',
    age: 40,
    isDev: true
}
console.log(`${person5.name} is a ${person5.isDev} Dev and is ${person5.age} years old`);

```

Listado 2-18 Ejecución del código

Ahora, en el localhost, puedes ver el registro de la consola deseado (véase la Figura **2-5**).



Figura 2-5: La consola

Resumen: En este capítulo, has aprendido sobre los diferentes tipos en TypeScript. Has aprendido sobre el tipo `numérico` y, a continuación, has pasado a los tipos de `cadena` y `booleano`. Tras aprender sobre los tipos de objeto y matriz, has aprendido sobre las matrices complejas. A continuación, has aprendido sobre los tipos de unión, literales, enumeraciones y opcionales. El capítulo ha concluido con un análisis de las interfaces.

3. El compilador de TypeScript

Cada vez que ejecutas el archivo de TypeScript, estás realizando cambios en él. En este capítulo, aprenderás otras formas de supervisar y realizar cambios.

Modo de vigilancia

Puedes supervisar los cambios en el archivo `main.js` utilizando el modo de supervisión. De este modo, no tendrás que ejecutar el archivo después de cada cambio. Debes ejecutar el comando `tsc` con el indicador `-w`, de la siguiente manera:

```
1 tsc main.ts -w
```

Si a continuación añades algo al archivo, se convertirá en el archivo JavaScript correspondiente. Véase el Listado **3-1**.

```
type PersonNewName = string;
const person6: PersonNewName = 'Mousam';
console.log(`New Developer is ${person6}`);
```

Listado 3-1: Nuevo código

Puedes ver los nuevos cambios en el localhost (véase la Figura **3-1**).

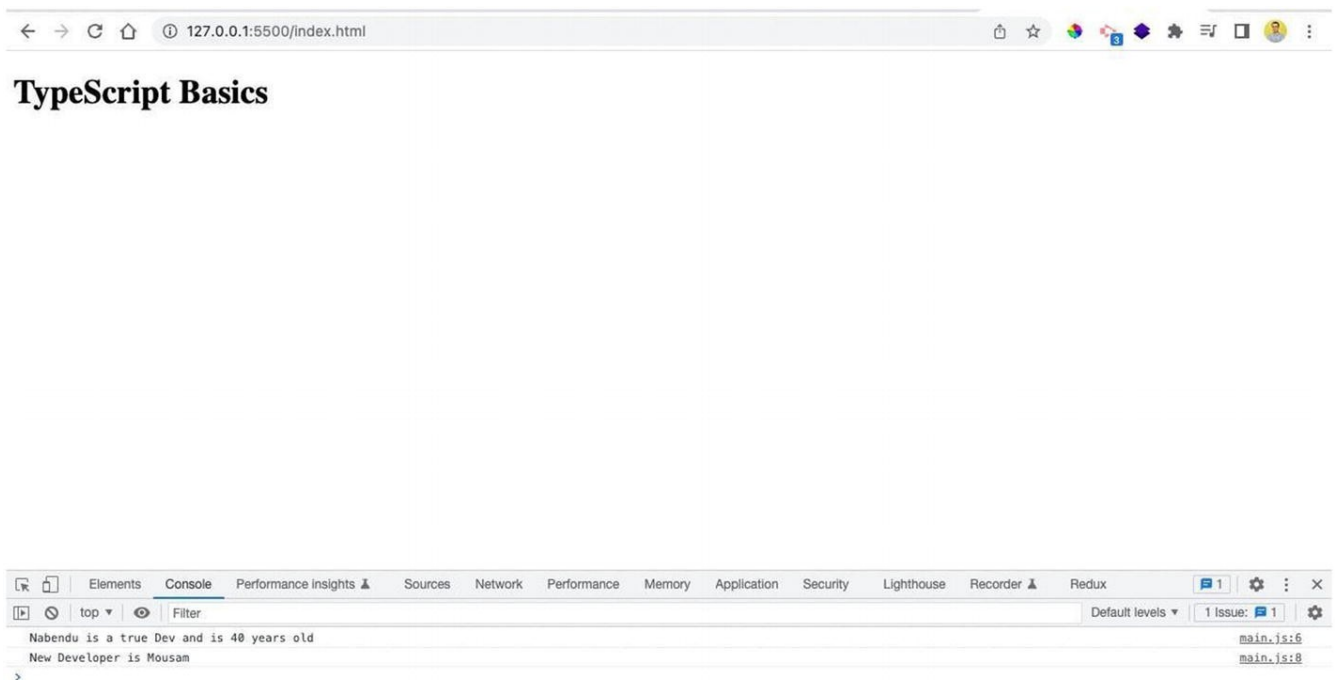


Figura 3-1: Cambios locales

Compilación de un proyecto completo

Para compilar un proyecto completo, hay que convertirlo en un proyecto de TypeScript. Para ello, hay que ejecutar el comando `tsc -init`, que creará un archivo `tsconfig.json` en el directorio raíz. Véase la Figura **3-2**.

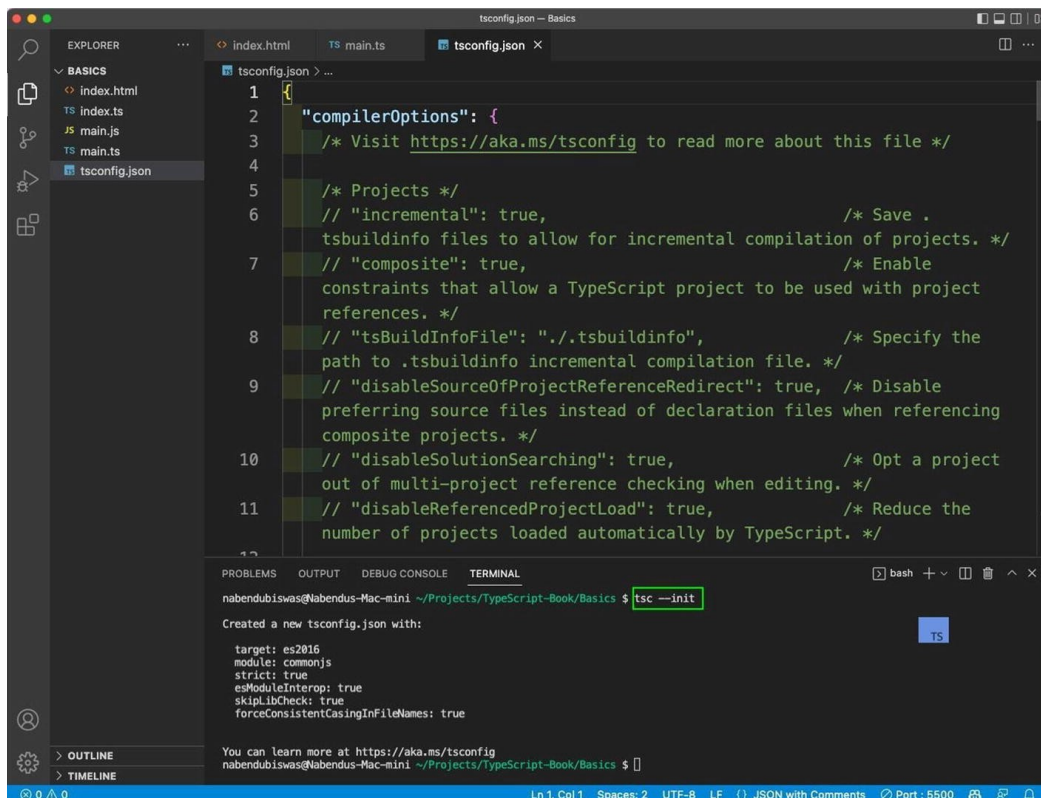


Figura 3-2: Compilación de un proyecto

A continuación, puedes ejecutar el comando `tsc` para compilar todos los archivos de TypeScript. Sin embargo, el archivo `index.ts` contiene muchos errores. La figura **3-3** muestra uno de estos errores.

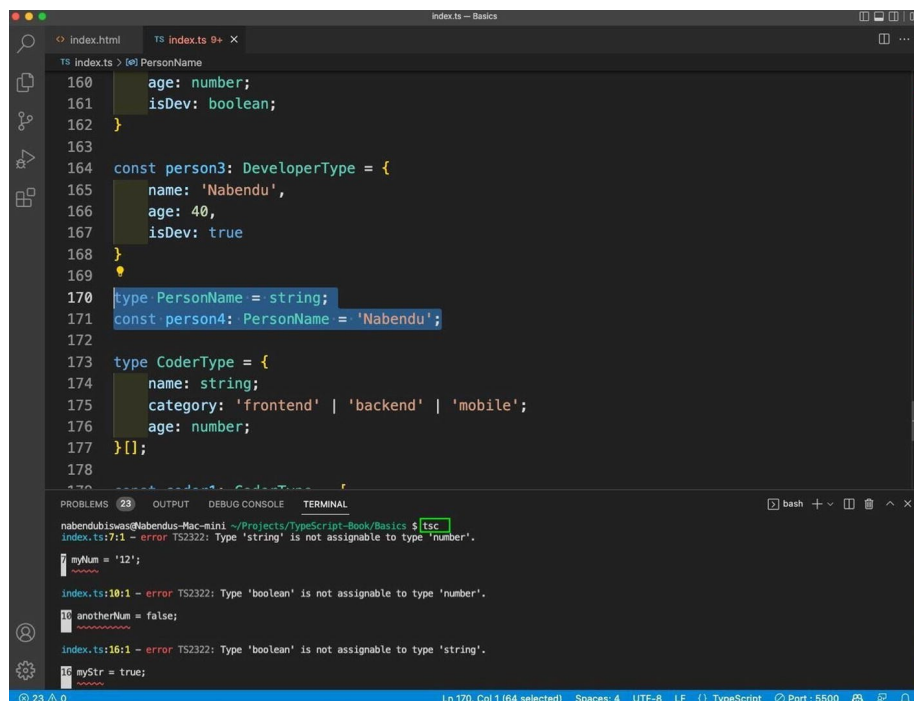
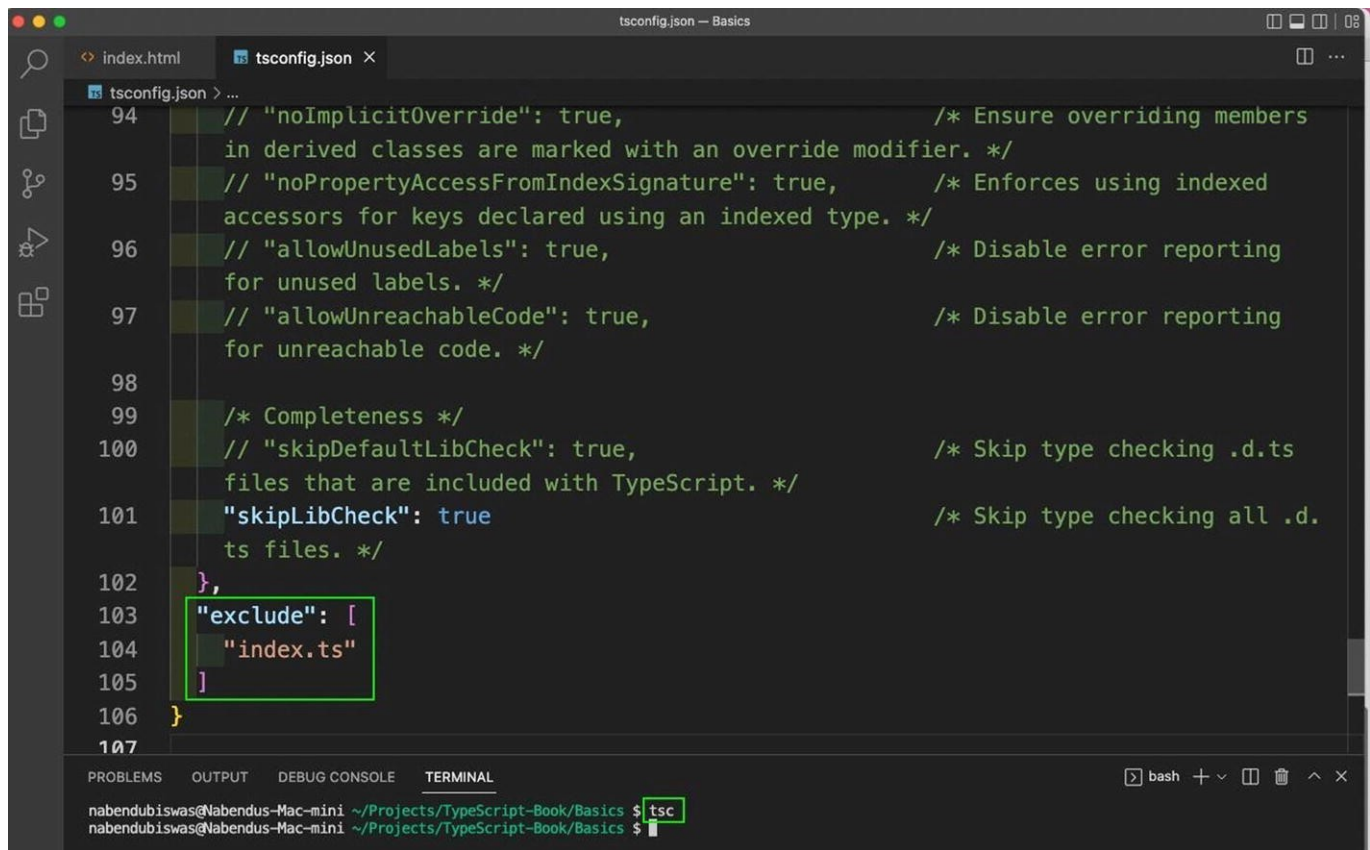


Figura 3-3: El error de tsc

Debes excluir este archivo añadiendo una entrada «exclude» al archivo `tsconfig.json`. Ahora, cuando ejecutes el comando `tsc`, no aparecerá ningún error (véase la Figura **3-4**).



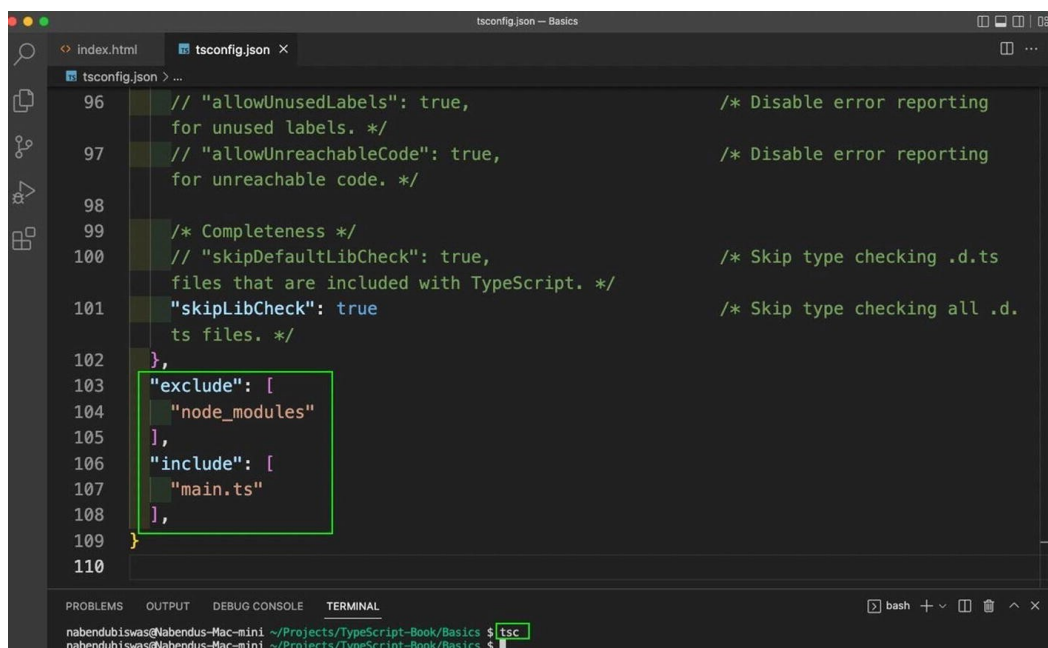
```
tsconfig.json — Basics
index.html  tsconfig.json x
tsconfig.json > ...
94 // "noImplicitOverride": true, /* Ensure overriding members
    in derived classes are marked with an override modifier. */
95 // "noPropertyAccessFromIndexSignature": true, /* Enforces using indexed
    accessors for keys declared using an indexed type. */
96 // "allowUnusedLabels": true, /* Disable error reporting
    for unused labels. */
97 // "allowUnreachableCode": true, /* Disable error reporting
    for unreachable code. */
98
99 /* Completeness */
100 // "skipDefaultLibCheck": true, /* Skip type checking .d.ts
    files that are included with TypeScript. */
101 "skipLibCheck": true /* Skip type checking all .d.
    ts files. */
102 },
103 "exclude": [
104   "index.ts"
105 ],
106 }
107

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
nabendubiswas@Nabendus-Mac-mini ~/Projects/TypeScript-Book/Basics $ tsc
nabendubiswas@Nabendus-Mac-mini ~/Projects/TypeScript-Book/Basics $
```

Figura 3-4 El archivo tsc de nuevo, sin el error

También puedes excluir todos los «node_modules», que se crean al utilizar una biblioteca de terceros. A veces contienen código TypeScript y no quieres compilarlos.

Al igual que con la opción «exclude», hay que utilizar «include» para añadir los archivos mencionados. En el ejemplo de la figura [3-5](#), se incluye el archivo main.ts.



```
tsconfig.json — Basics
index.html  tsconfig.json x
tsconfig.json > ...
96 // "allowUnusedLabels": true, /* Disable error reporting
    for unused labels. */
97 // "allowUnreachableCode": true, /* Disable error reporting
    for unreachable code. */
98
99 /* Completeness */
100 // "skipDefaultLibCheck": true, /* Skip type checking .d.ts
    files that are included with TypeScript. */
101 "skipLibCheck": true /* Skip type checking all .d.
    ts files. */
102 },
103 "exclude": [
104   "node_modules"
105 ],
106 "include": [
107   "main.ts"
108 ],
109 }
110

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL
nabendubiswas@Nabendus-Mac-mini ~/Projects/TypeScript-Book/Basics $ tsc
nabendubiswas@Nabendus-Mac-mini ~/Projects/TypeScript-Book/Basics $
```

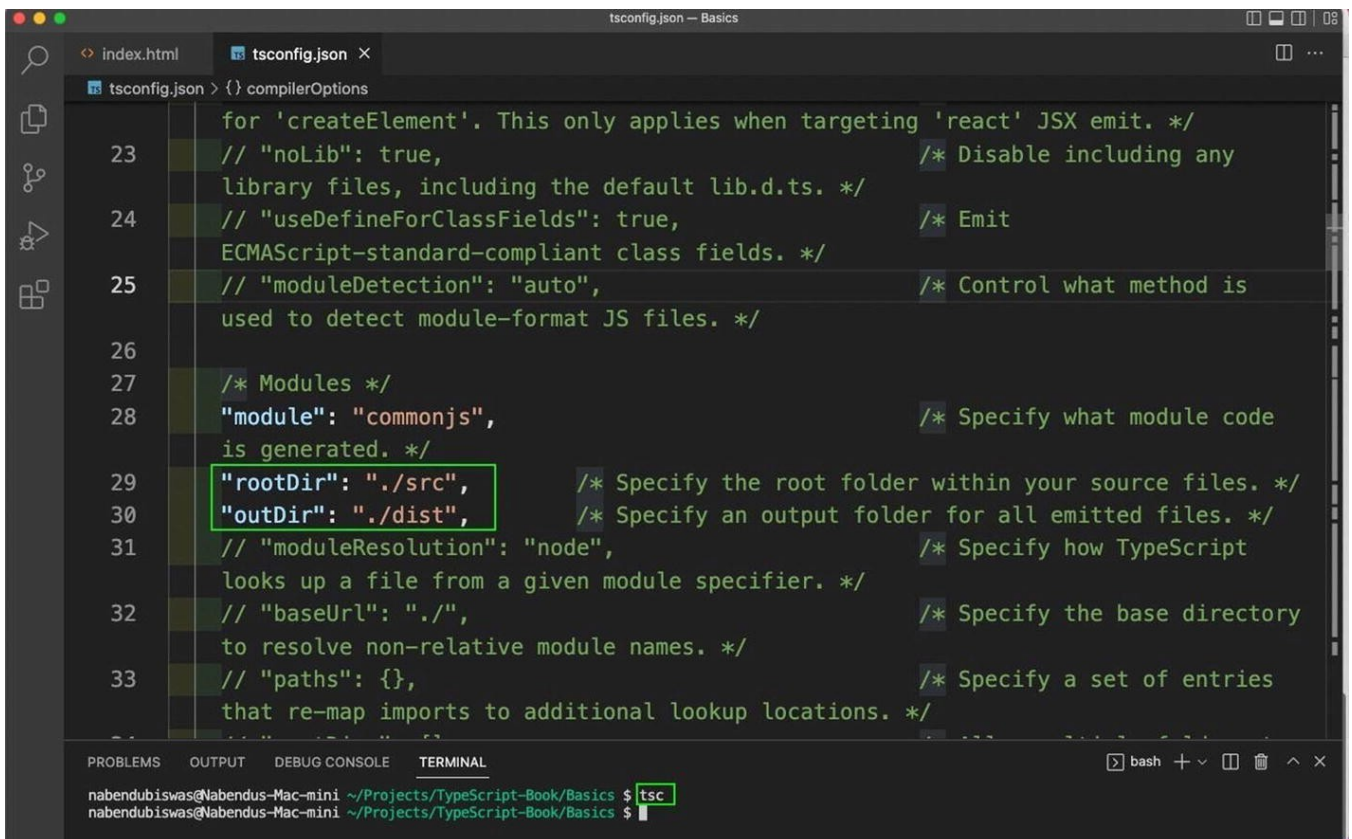
Figura 3-5: Esta instrucción «include» añade el archivo main.ts

rootDir y outDir

Para organizar tu proyecto de TypeScript, debes guardar todos los archivos de TypeScript en una carpeta `src`.

Has colocado los archivos `index.ts` y `main.ts` en la carpeta `src`. Crea también una carpeta `dist`.

En el archivo `tsconfig.json`, descomenta `rootDir` y `outDir`. Aquí es donde debes indicar las carpetas correspondientes (véase la Figura [3-6](#)). También debes ejecutar el comando `tsc`; no debería dar ningún error.



```
tsconfig.json > {} compilerOptions
  for 'createElement'. This only applies when targeting 'react' JSX emit. */
23 // "noLib": true,                      /* Disable including any
    library files, including the default lib.d.ts. */
24 // "useDefineForClassFields": true,      /* Emit
    ECMAScript-standard-compliant class fields. */
25 // "moduleDetection": "auto",           /* Control what method is
    used to detect module-format JS files. */
26
27 /* Modules */
28 "module": "commonjs",                  /* Specify what module code
    is generated. */
29 "rootDir": "./src",                    /* Specify the root folder within your source files. */
30 "outDir": "./dist",                   /* Specify an output folder for all emitted files. */
31 // "moduleResolution": "node",          /* Specify how TypeScript
    looks up a file from a given module specifier. */
32 // "baseUrl": ".",                     /* Specify the base directory
    to resolve non-relative module names. */
33 // "paths": {},                        /* Specify a set of entries
    that re-map imports to additional lookup locations. */
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL

```
nabendubiswas@Nabendus-Mac-mini ~/Projects/TypeScript-Book/Basics $ tsc
nabendubiswas@Nabendus-Mac-mini ~/Projects/TypeScript-Book/Basics $
```

Figura 3-6 El archivo `tsconfig.json`

Ahora puedes ver el archivo `main.js` en la carpeta `dist` (véase la Figura [3-7](#)). También debes cambiar la ruta del archivo `main.js` en el archivo `index.html`.

```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta http-equiv="X-UA-Compatible" content="IE=edge">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <title>TypeScript Basics</title>
8 </head>
9 <body>
10  <h1>TypeScript Basics</h1>
11  <script src="dist/main.js"></script>
12 </body>
13 </html>
```

Figura 3-7 El archivo index.html

Resumen: En este capítulo, has aprendido a configurar correctamente tus proyectos de TypeScript.